

МІНІСТЕРСТВО ОСВІТИ ТА НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ
ІМЕНІ ІГОРЯ СІКОРСЬКОГО»
КАФЕДРА ОБЧИСЛЮВАЛЬНОЇ ТЕХНІКИ

Лабораторна робота №6

**Тема: «ПРИСТРОЇ ДЛЯ ПЕРЕТВОРЕННЯ ЧИСЕЛ.
ТИПОВІ ВУЗЛИ КОМП'ЮТЕРА»**

Виконав:

студент групи ІО-43

Семчук Тарас Русланович

Номер залікової книжки: 4315

Перевірів роботу:

Нікольський С. С.

Київ 2026

Лабораторна робота No 6

ПРИСТРОЇ ДЛЯ ПЕРЕТВОРЕННЯ ЧИСЕЛ. ТИПОВІ ВУЗЛИ КОМП'ЮТЕРА

Порядок виконання роботи

Мій варіант:

$4315_{10} - 1000011011011_2$

$h_6=0, h_5=1, h_4=1, h_3=0, h_2=1, h_1=1$

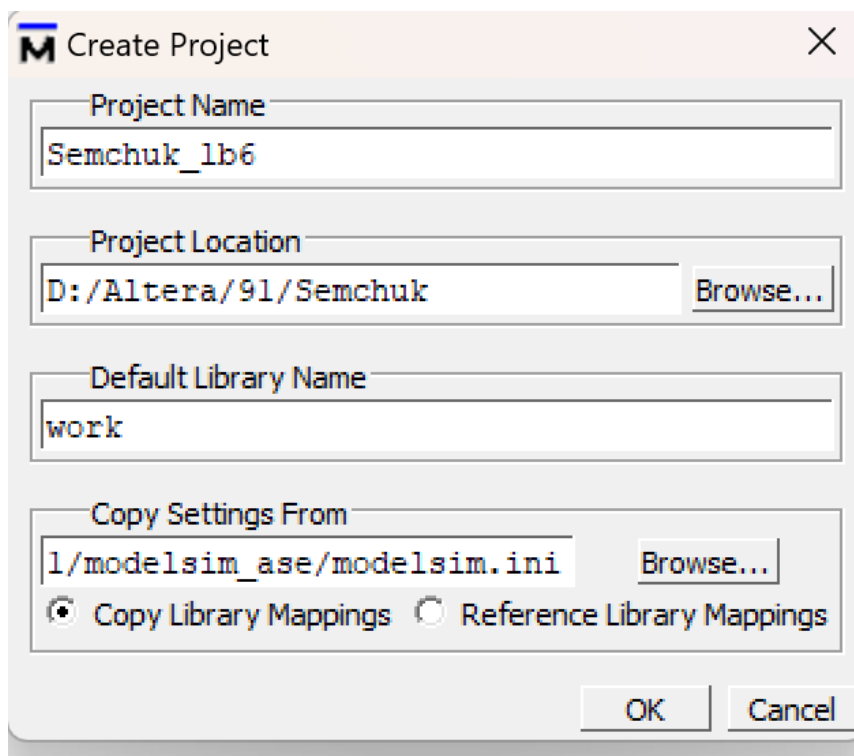
$h_2 \ h_4 \ h_1 - 111$

Тоді:

111	6
-----	---

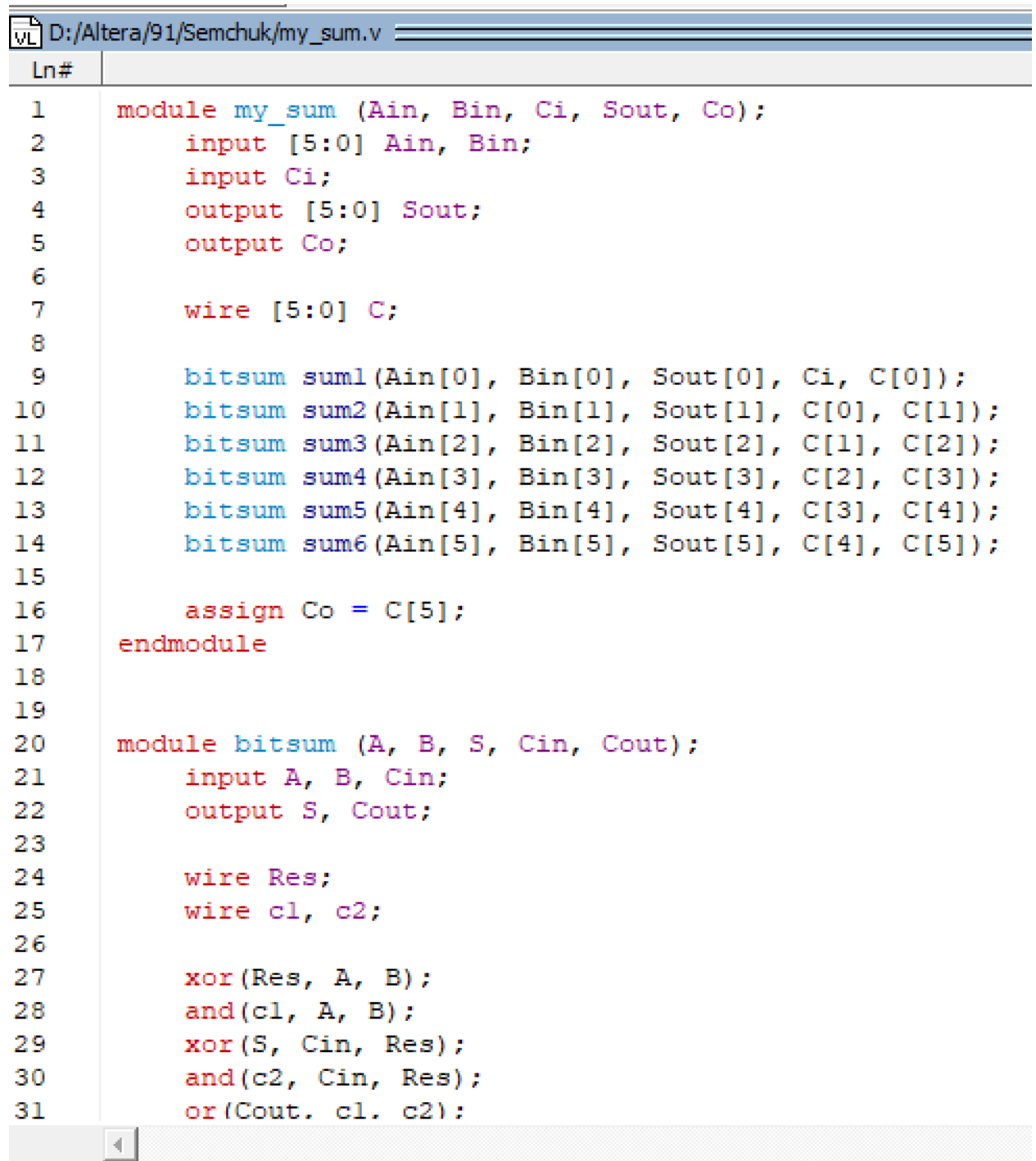
6 розрядів

Створюю проект новий та файли в ньому:



Project - D:/Altera/91/Semchuk/Semchuk_lb6					
Name	Status	Type	Order	Modified	
 test_sum.v		Verilog	2	05/03/26 03:52:32 PM	
 ref_sum.v		Verilog	1	05/03/26 03:52:22 PM	
 my_sum.v		Verilog	0	05/03/26 03:52:12 PM	

Далі вставляю код в ці файли по черзі:



The screenshot shows a Verilog code editor with a file named `D:/Altera/91/Semchuk/my_sum.v`. The code is organized into two modules. The first module, `my_sum`, defines inputs `Ain`, `Bin`, and `Ci`, and outputs `Sout` and `Co`. It uses a 6-bit carry register `C` and six `bitsum` sub-modules to calculate the sum of `Ain` and `Bin` bit-by-bit, propagating the carry from `Ci` through `C[0]` to `C[5]`, which is then assigned to `Co`. The second module, `bitsum`, is a 1-bit full adder that takes inputs `A`, `B`, and `Cin`, and produces outputs `S` (sum) and `Cout` (carry-out). It uses basic logic gates: XOR for the sum and AND/OR for the carry propagation.

```
1  module my_sum (Ain, Bin, Ci, Sout, Co);
2      input [5:0] Ain, Bin;
3      input Ci;
4      output [5:0] Sout;
5      output Co;
6
7      wire [5:0] C;
8
9      bitsum sum1(Ain[0], Bin[0], Sout[0], Ci, C[0]);
10     bitsum sum2(Ain[1], Bin[1], Sout[1], C[0], C[1]);
11     bitsum sum3(Ain[2], Bin[2], Sout[2], C[1], C[2]);
12     bitsum sum4(Ain[3], Bin[3], Sout[3], C[2], C[3]);
13     bitsum sum5(Ain[4], Bin[4], Sout[4], C[3], C[4]);
14     bitsum sum6(Ain[5], Bin[5], Sout[5], C[4], C[5]);
15
16     assign Co = C[5];
17 endmodule
18
19
20 module bitsum (A, B, S, Cin, Cout);
21     input A, B, Cin;
22     output S, Cout;
23
24     wire Res;
25     wire c1, c2;
26
27     xor(Res, A, B);
28     and(c1, A, B);
29     xor(S, Cin, Res);
30     and(c2, Cin, Res);
31     or(Cout, c1, c2);
```

Наступний файл:

Altera/91/Semchuk/ref_sum.v

```
module ref_sum (Ain, Bin, Ci, Sout, Co);
    input [5:0] Ain, Bin;
    input Ci;
    output [5:0] Sout;
    output Co;

    reg [6:0] S;

    always @(Ain, Bin, Ci)
        S = Ain + Bin + Ci;

    assign Sout = S[5:0];
    assign Co = S[6];
endmodule
```

Наступний код:

D:/Altera/91/Semchuk/test_sum.v

```
Ln#
1  `timescale 1ns/1ps
2
3  module test_sum;
4
5  wire Ci, cm, cr;
6  wire [5:0] Ain, Bin;
7  wire [5:0] res_my, res_ref;
8
9  reg [5:0] Ain_r, Bin_r;
10 reg Ci_r;
11
12 my_sum my_block (
13     .Ain(Ain),
14     .Bin(Bin),
15     .Ci(Ci),
16     .Sout(res_my),
17     .Co(cm)
18 );
19
20 ref_sum ref_block (
21     .Ain(Ain),
22     .Bin(Bin),
23     .Ci(Ci),
24     .Sout(res_ref),
25     .Co(cr)
26 );
27
28 initial begin
29     $display("Time Ain Bin Ci res_my cm res_ref cr");
30     $monitor($time, " ", Ain, " ", Bin, " ", Ci, " ", res_my, " ", cm, " ", res_ref, " ", cr);
31     #400 $finish;
32 end
```

Запускаю компіляцію всіх цих файлів:

Project - D:/Altera/91/Semchuk/Semchuk_lb6				
Name	Sta ▾	Type	Order	Modified
 my_sum.v	✓	Verilog	0	05/03/26 03:56:54 PM
 test_sum.v	✓	Verilog	2	05/03/26 04:23:05 PM
 ref_sum.v	✓	Verilog	1	05/03/26 04:21:38 PM

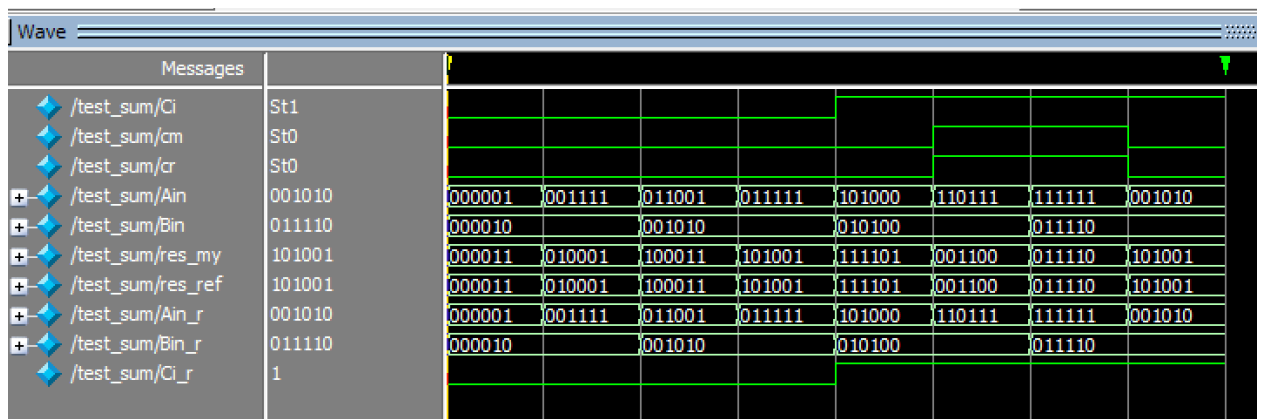
Тепер запускаю симуляцію:

Name	Type	Path
 work	Library	D:/Altera/91/Semchuk/work
 bitsum	Module	D:/Altera/91/Semchuk/my_sum.v
 demux1to8	Module	D:/Altera/91/Semchuk/demux1to8.v
 my_sum	Module	D:/Altera/91/Semchuk/my_sum.v
 ref_sum	Module	D:/Altera/91/Semchuk/ref_sum.v
 tb_demux	Module	D:/Altera/91/Semchuk/tb_demux.v
 test_sum	Module	D:/Altera/91/Semchuk/test_sum.v

І в результаті отримую ось таке в консолі:

```
VSIM 3> run 400ns
# Time Ain Bin Ci res_my cm res_ref cr
#           0   1   2   0   3   0   3   0
#          50  15   2   0  17   0  17   0
#         100  25  10   0  35   0  35   0
#         150  31  10   0  41   0  41   0
#         200  40  20   1  61   0  61   0
#         250  55  20   1  12   1  12   1
#         300  63  30   1  30   1  30   1
#         350  10  30   1  41   0  41   0
# ** Note: $finish      : D:/Altera/91/Semchuk/test_sum.v(31)
#   Time: 400 ns  Iteration: 0  Instance: /test_sum
```

І часові діаграми:



Тепер проаналізуємо цю часову діаграму і напишемо висновок:

У цій лабораторній роботі я реалізував 6-розрядний суматор мовою Verilog. Я створив структурний модуль суматора, а також еталонну модель і testbench для перевірки роботи. Після цього виконав моделювання в програмі ModelSim. У результаті перевірки було видно, що вихідні дані обох моделей співпадають, а також правильно формується сигнал переносу. Отже, розроблений суматор працює правильно і відповідає поставленому завданню.